

MCP Command-Reference Server

Implementation plan — turning coder_commands.xlsx into an MCP server queryable from Claude Code

1. Source Data

coder_commands.xlsx currently has 14 sheets (mongo, tmux, linux, sql, docker, podman, shell, git, ansible, python, jython, uPython, ai, postgres), 7–133 rows each, collected over the last two years. Column headers are inconsistent across sheets (COMMAND/MEANING/ADDITIONAL, COMMAND/RESULT, COMMAND/COMMENT/ACTION, COMMAND/ACTION), some sheets have no header row at all, and several (docker, ansible, postgres) use section-divider rows such as “IMAGE MANAGEMENT” as pseudo-headers. This irregularity is the main technical risk in the project and drives the need for a dedicated normalization step rather than a direct read of the spreadsheet.

2. Decisions You Made, and My Opinion

Decision	Your choice
Data layer	Postgres, with the xlsx syncing in as the editing surface.
Tool design	Both — one generic full-text search tool plus a per-topic browse tool for each sheet.
Read/write scope	Read-only for v1; no write-back to the database from Claude yet.
Docker topology	Two containers — MCP server and FastAPI gateway are separate services.
Sync trigger	File watcher — edits to the xlsx are picked up automatically, no manual step.

My take on each choice

Postgres over SQLite or plain files

Good call. It's heavier than this single-user tool strictly needs, but you're actively learning SQL and Postgres, so the extra container is doing double duty as practice. If this ever felt like too much operational weight for what it does, SQLite would be the natural fallback — not needed now.

Both tool shapes (search + per-topic browse)

Sensible middle ground. The generic search_commands tool is what makes natural-language questions like “insert a new row into a table” work well; the per-topic browse tools are a cheap addition since they can be generated from the distinct topics already in the database rather than hand-written. Worth revisiting only if the topic count grows large enough to clutter the tool list.

Read-only for v1

The right scope call. It keeps the first version's surface area small — no write validation, no conflict handling between Excel edits and Claude-initiated writes — and defers that complexity until the read path is proven.

Adding an `add_command` tool later is a natural v2, not a redesign.

Two containers (MCP server + FastAPI)

This matches what you originally described and gives you a real gateway boundary for auth or logging later, which a single merged process wouldn't. The tradeoff is an extra network hop (FastAPI reverse-proxying to the MCP container) for a tool only you use — more infrastructure than strictly necessary, but reasonable given you're using this project to practice Docker/compose patterns as much as to solve the lookup problem itself.

File watcher for sync

Correctly chosen given your requirement that spreadsheet edits must reach the MCP server without a manual step. One caveat worth flagging: Docker Desktop bind-mount file events on Windows are unreliable, so the watcher should poll the file's modified time (every 5–10s) rather than rely on inotify-style events. That means a short delay after saving, not instant propagation — acceptable for this use case.

3. Implementation Plan

- 1. Repo scaffolding — Create the project structure (`sync/`, `mcp_server/`, `api/`, `docker-compose.yml`, `.env.example`) matching the four-service topology below.
- 2. Postgres schema — Write `schema.sql` defining the `commands` table (`topic`, `section`, `command`, `description`, `extra`, `row_order`) with a generated `tsvector` column and GIN index for full-text search.
- 3. Ingest / normalization module — Parse each sheet with `openpyxl`, detect header vs. no-header rows, detect section-divider rows (single populated cell) and carry them forward as the section value, and canonicalize every sheet into the common row shape.
- 4. Sync service (ingest + watcher) — Combine the ingest module with a polling watcher on the `xlsx` file's modified time. On change, run a per-topic transactional replace (delete rows for that topic, reinsert) so edits, reorders, and deletions are all handled the same way.
- 5. MCP server — Implement `search_commands(query, topic?, limit)`, `list_topics()`, and one `browse_tool` generated per distinct topic at startup. Serve over Streamable HTTP.
- 6. FastAPI gateway — Stand up the FastAPI container and reverse-proxy `/mcp/*` to the MCP server container over the compose network, as the integration point for future auth/logging.
- 7. Docker Compose wiring — Wire postgres, sync, mcp-server, and fastapi together with a shared network, a read-only bind mount of the `xlsx` into the sync service, environment variables, and healthchecks so dependent services wait for Postgres to be ready.
- 8. Local verification — Bring the stack up, confirm the sync service populates Postgres correctly, confirm `search_commands` returns sensible ranked results, and confirm the FastAPI proxy passes MCP traffic through correctly.
- 9. Claude Code integration — Register the server with `claude mcp add --transport http`, then run a set of real natural-language queries end-to-end (e.g. “how do I insert a new row into a table”) to confirm the tool selection and results are useful in practice.
- 10. Documentation — Short README covering how to add a new topic sheet, how the sync/watcher works, and how to restart or rebuild individual services.

Next step: confirm this plan, then scaffolding begins with the repo structure and Postgres schema.